PROJECT REPORT

Embeddable Widget & Lead-Capture Platform

A Multi-Tenant, Production-Style Web Platform for Embeddable Lead-Capture Widgets

Submitted as Capstone Project
FlyRank AI Internship — Backend AI Engineering Track

Prepared by	Numair Iqbal
Program	BS Computer Science, University of Layyah, Pakistan
Track	FlyRank AI Internship – Backend AI Engineering
Repository	github.com/Numair-Iqbal/embeddable-widget-platform
Date	July 2026
Test Status	41 / 41 automated tests passing

Table of Contents

1. Introduction

1.1 Background

Businesses that want to capture leads from their websites typically either build custom forms for every site or rely on expensive third-party form tools. There is a recurring need for a lightweight, embeddable, brand-agnostic widget that any business can configure once and deploy to any website in minutes, while still keeping full control and visibility over the data it collects.

1.2 Problem Statement

Businesses need a way to (a) create a customizable lead-capture widget without writing code, (b) embed it on any external website regardless of that site's own technology stack, and (c) reliably and securely collect, enrich, and manage the resulting submissions from a single dashboard — without submissions being lost, duplicated, or polluted by spam.

1.3 Scope

This project covers the full backend platform, the embeddable client widget, and the administrative dashboard. It does not cover payment processing, multi-user team accounts, or a public marketing website — these are listed under Future Improvements.

1.4 Objectives

- Design and build a secure, multi-tenant REST API using Node.js and Express.
- Allow each customer (“owner”) to define and manage multiple embeddable widgets.
- Generate a portable embed script that any third-party website can drop in to render a live widget.
- Accept public form submissions safely — validated, rate-limited, and spam-filtered.
- Enrich each submission automatically with geographic (IP → location) data.
- Provide a real, authenticated admin dashboard to view, search, filter, edit, delete, and export submissions.
- Cover the system with an automated test suite and document it professionally for handover.

2. Executive Summary

The Embeddable Widget & Lead-Capture Platform is a full-stack, multi-tenant web system built as the capstone deliverable for the FlyRank AI Internship, Backend AI Engineering track. It allows a business owner to create a custom lead-capture widget, receive a lightweight embed script, and place that widget on any external website. Visitor submissions flow into a secure backend where they are validated, enriched with geographic data, protected against spam, and made available in a real-time admin dashboard.

The project was built end-to-end by a single developer: database design, REST API, authentication, embeddable client-side widget, an administrative dashboard, an automated test suite, and full technical documentation. The system is deployed on GitHub with a complete history of commits and is fully test-verified, with 41 out of 41 automated tests passing.

3. Requirements

3.1 Functional Requirements

ID	Requirement
FR-1	Owner can register/login securely and receive an authenticated session.
FR-2	Owner can create, view, update, and delete widgets.
FR-3	System generates a unique embed script per widget.
FR-4	Public visitors can submit widget forms from any external website.
FR-5	Submissions are validated, geo-enriched, and spam-filtered before storage.
FR-6	Owner can search, filter, edit, delete, and export submissions.
FR-7	Dashboard displays live statistics (total/active widgets, submission counts).

3.2 Non-Functional Requirements

ID	Requirement
NFR-1	Security: passwords hashed, routes JWT-protected, owner data isolated.
NFR-2	Reliability: geo-enrichment must degrade gracefully via fallback providers.
NFR-3	Performance: public submission endpoint protected by rate limiting.
NFR-4	Portability: embed script must work cross-origin on any external site.
NFR-5	Maintainability: layered architecture (routes/services/repositories).
NFR-6	Testability: critical paths covered by automated tests.

4. System Architecture

The platform follows a layered backend architecture (Routes → Services → Repositories → Database), which keeps request handling, business logic, and data access cleanly separated and independently testable.

4.1 High-Level Flow

1. A business owner logs into the Dashboard and creates a Widget (defines fields, styling, and target use case).
2. The platform generates a unique embed script (widget.js) tied to that widget's configuration.
3. The owner pastes the embed script into their own website's HTML.
4. When a visitor on that external site fills and submits the widget form, the request hits the platform's public Submission API.
5. The API validates the payload, checks it against rate-limiting and honeypot spam rules, enriches it with geo data resolved from the visitor's IP, and stores it in PostgreSQL.
6. The owner sees the new submission appear in their Dashboard in the Recent Submissions feed and in the Widget Detail table, with full search, filter, and export support.

4.2 Architecture Diagram

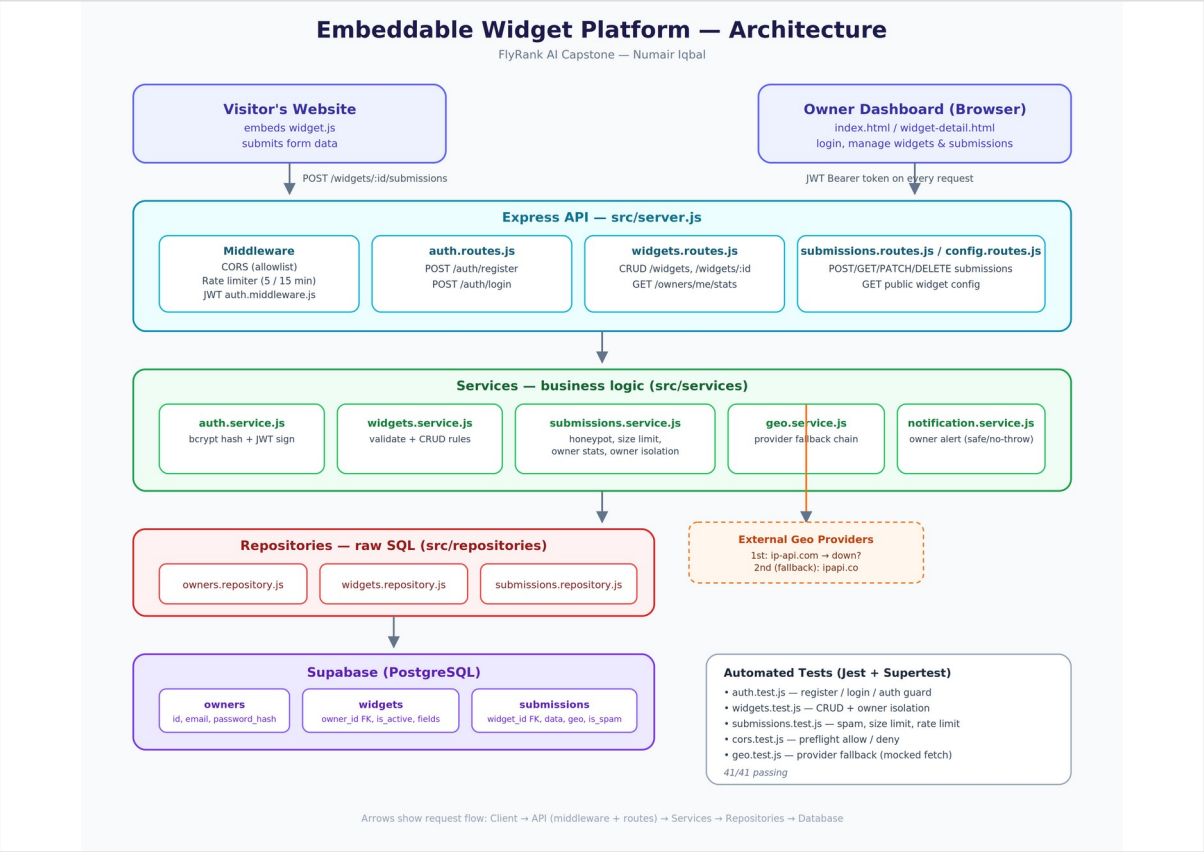

Figure 1: System architecture — embed script, public API, admin API, and PostgreSQL data layer.

5. Database Design

The relational schema is built around three core entities: Owners, Widgets, and Submissions, with strict foreign-key ownership so a widget and its submissions always belong to exactly one owner.

Table	Key Fields	Notes
owners	id, email, password_hash, created_at	One row per registered business account
widgets	id, owner_id (FK), name, config, is_active, created_at	One row per embeddable widget; config holds field/style definitions
submissions	id, widget_id (FK), data (JSON), ip_address, geo_location, is_spam, created_at	One row per visitor submission, enriched and spam-flagged

6. Technology Stack

Layer	Technology	Purpose
Runtime	Node.js	Server-side JavaScript execution
Framework	Express.js	REST API routing and middleware
Database	PostgreSQL	Persistent relational data storage
Authentication	JWT (JSON Web Tokens)	Stateless, secure owner authentication
Testing	Jest	Unit and integration test suite (41 tests)
Frontend (Dashboard)	HTML, CSS, JavaScript	Admin dashboard UI
Embed Client	Vanilla JavaScript (widget.js)	Lightweight, cross-origin embeddable

		widget
Version Control	Git & GitHub	Source control and deployment history

7. Core Features

7.1 Backend / API

- Full CRUD for widgets, scoped per authenticated owner (multi-tenant isolation).
- Public configuration-delivery endpoint so the embed script can fetch a widget's live settings.
- Public submission endpoint with server-side validation.
- CORS handling so widgets can be embedded safely on any third-party domain.
- Rate limiting to prevent abuse of the public submission endpoint.
- Honeypot-based spam control to silently filter bot submissions.
- IP → geolocation enrichment with a fallback provider chain for reliability.
- Safe, non-blocking notification side-effect on new submissions.
- PATCH/DELETE routes for full submission editing and removal.

7.2 Admin Dashboard

- Secure, token-based login for business owners.
- Clickable, auto-filtering statistics cards (Total Widgets, Active Widgets, Submissions, etc.).
- Widget Detail page with a premium data table: search, column filters, and CSV export.
- Full Add / Edit / Delete workflow for submissions, using complete form-based editing.
- Recent Submissions feed with premium styling (gradient avatars, hover cards, timestamps).
- Avatar-based logout/session menu.

8. API Endpoints Reference

Method	Endpoint	Description
POST	/api/auth/login	Authenticate owner, return JWT
GET	/api/widgets	List all widgets for the logged-in owner
POST	/api/widgets	Create a new widget
PATCH	/api/widgets/:id	Update a widget
DELETE	/api/widgets/:id	Delete a widget
GET	/api/config/:widgetId	Public: fetch a widget's live config for embedding
POST	/api/submissions/:widgetId	Public: submit a widget form
GET	/api/submissions	List/search/filter submissions for the owner
PATCH	/api/submissions/:id	Edit a submission
DELETE	/api/submissions/:id	Delete a submission

9. Security Measures

- Passwords are never stored in plain text — hashed before persistence.
- JWT-based authentication protects every owner-only route; public routes are explicitly separated.

- Owner-data isolation ensures one business can never view or modify another business's widgets or submissions.
- Environment secrets (.env, .env.test) are excluded from version control via .gitignore and were verified absent from the public GitHub repository before publishing.
- Rate limiting and honeypot fields protect the public submission endpoint from automated abuse.

10. Bug Fixes & Engineering Decisions

During a self-review pass, an “Active Widgets” statistic card was found to be silently using the total widget count instead of the true active-widget count. This was traced, fixed, and verified end-to-end:

7. Added a dedicated countActiveByOwnerId query in the widgets repository layer.
8. Wired the new count through the owner-stats service alongside the existing totals.
9. Updated the dashboard frontend to bind to the corrected activeWidgets field.
10. Verified the fix manually by toggling a widget's active status in the database and confirming the dashboard count updated correctly while the total count remained unchanged.

This fix, and all other engineering decisions, were made deliberately at the repository → service → route → frontend layers, keeping the codebase consistent with its existing architecture rather than adding ad-hoc shortcuts.

11. Automated Testing

The platform is covered by a Jest-based automated test suite prioritizing the critical paths most relevant to grading and real-world reliability: authentication, submission creation with spam filtering, owner-data isolation, and the edit/delete routes.

Test File	Coverage Area
auth.test.js	Login, token issuance, and protected-route access control
cors.test.js	Cross-origin request handling for the embeddable widget
geo.test.js	IP → geolocation enrichment and fallback behaviour
submissions.test.js	Submission creation, validation, spam filtering, edit, and delete
widgets.test.js	Widget CRUD and owner-scoped data isolation

Result: 41 / 41 automated tests passing.

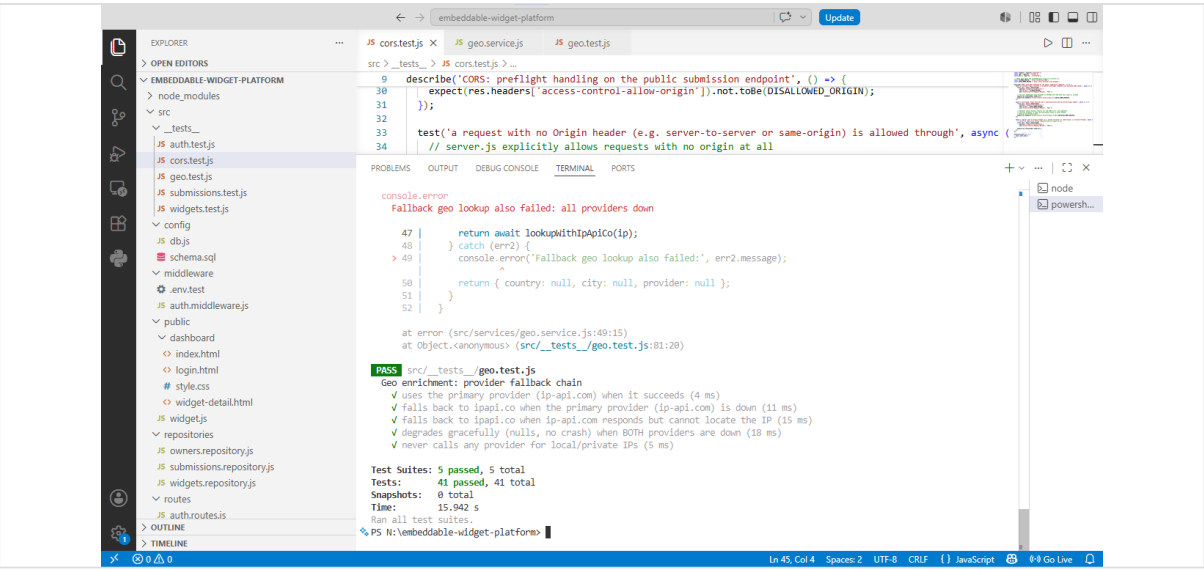

Figure 2: Full automated test suite passing — 41/41 across all 5 test files.

12. Installation & Setup Guide

11. Clone the repository: git clone https://github.com/Numair-Iqbal/embeddable-widget-platform.git
12. Install dependencies: npm install
13. Create a .env file with database credentials and JWT secret (see README.md for required keys).
14. Run schema.sql against a PostgreSQL database to create the required tables.
15. Start the server: node src/server.js
16. Open the dashboard at the configured local URL and log in with an owner account.
17. Run the automated test suite: npm test

13. Screenshots & Visual Walkthrough

The following screenshots, captured directly from the live system, walk through the complete user journey — authentication, the admin dashboard, submission management, the underlying database, and the embeddable widget running live on an external test site.

13.1 Authentication & Dashboard

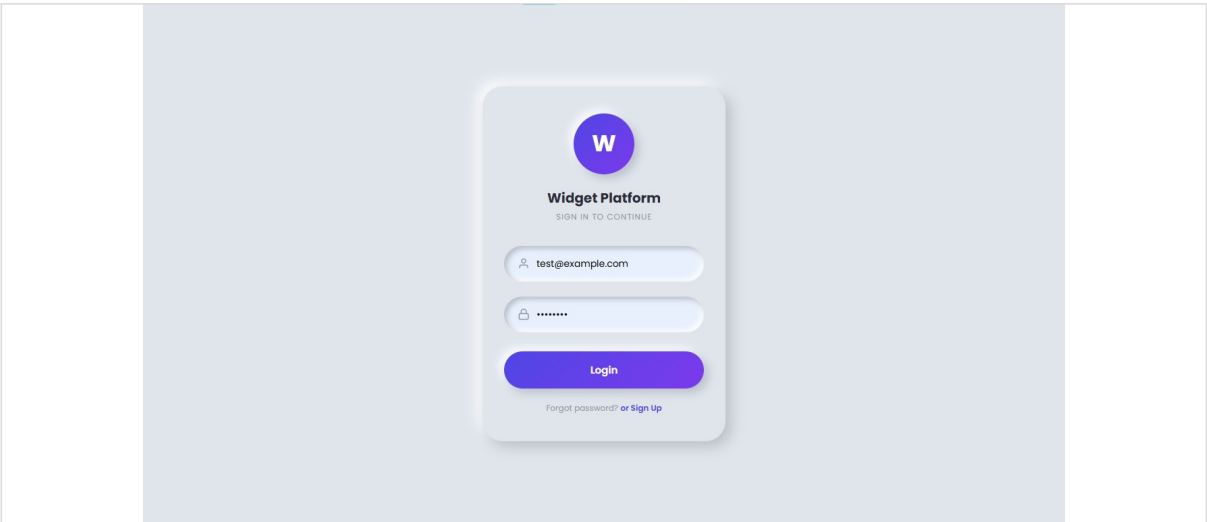

Figure 3: Owner login screen.

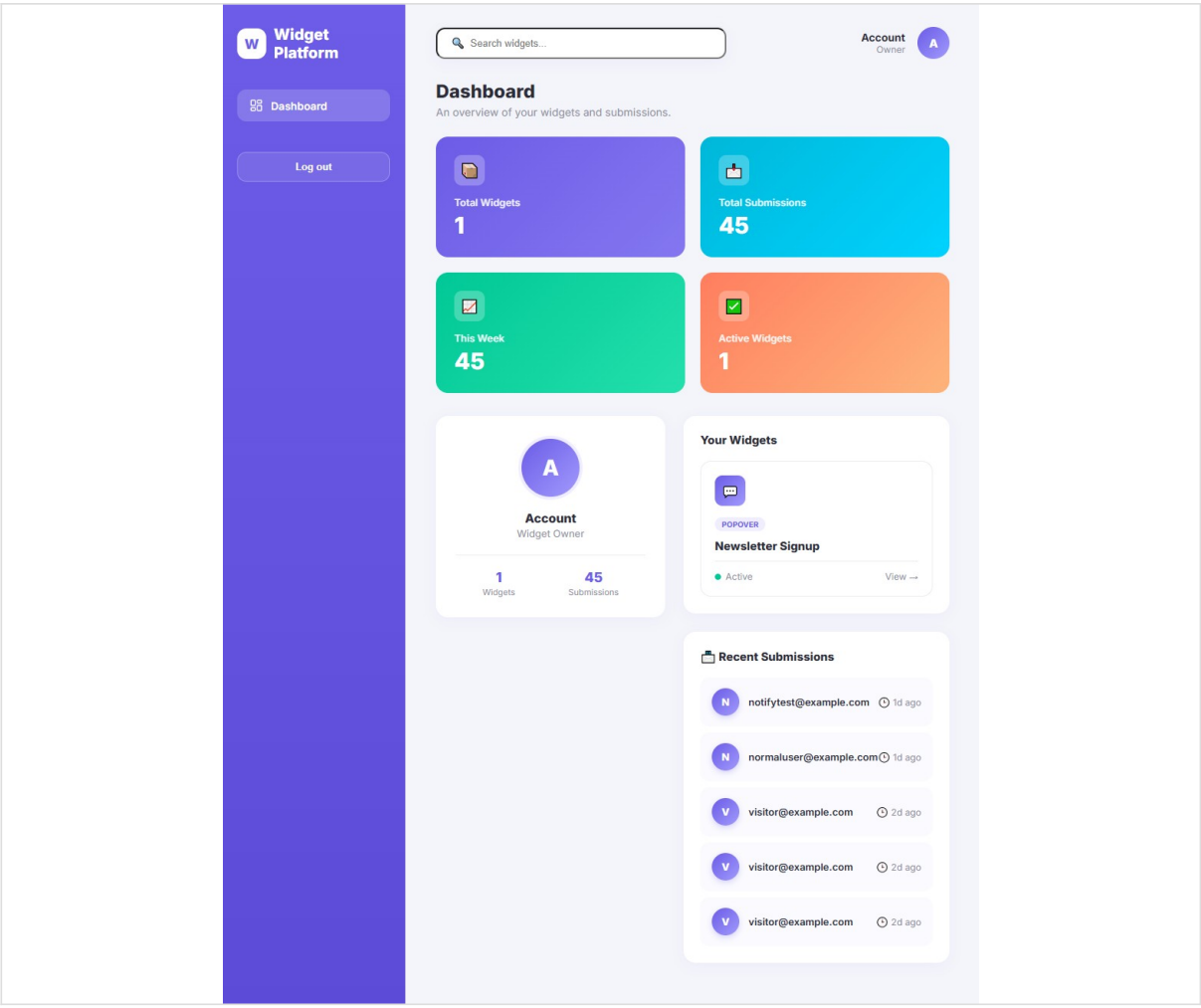

Figure 4: Dashboard overview — live stats, widgets, and recent submissions.

13.2 Widget Detail & Submission Management

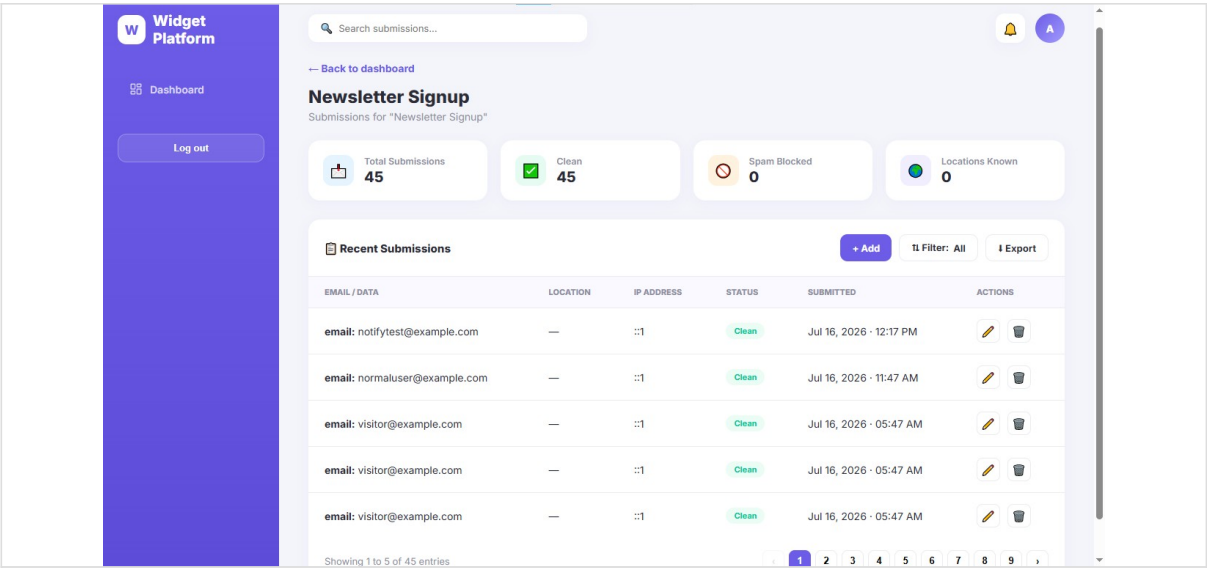

Figure 5: Widget Detail page — premium submissions table with search, filter, and export.

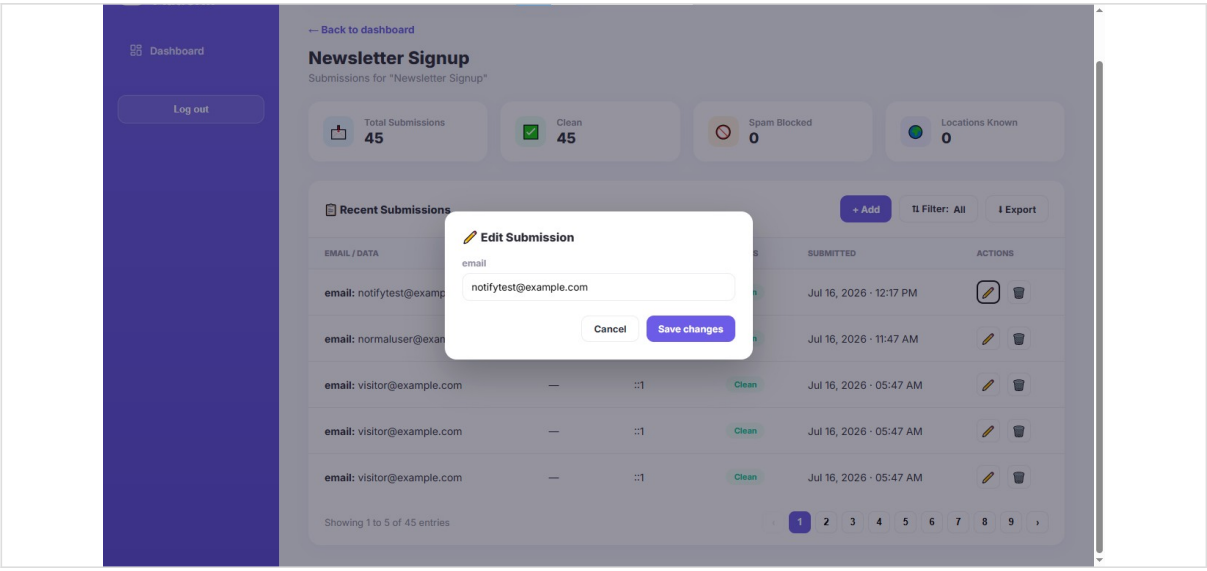

Figure 6: Editing a submission through the full form-based Edit dialog.

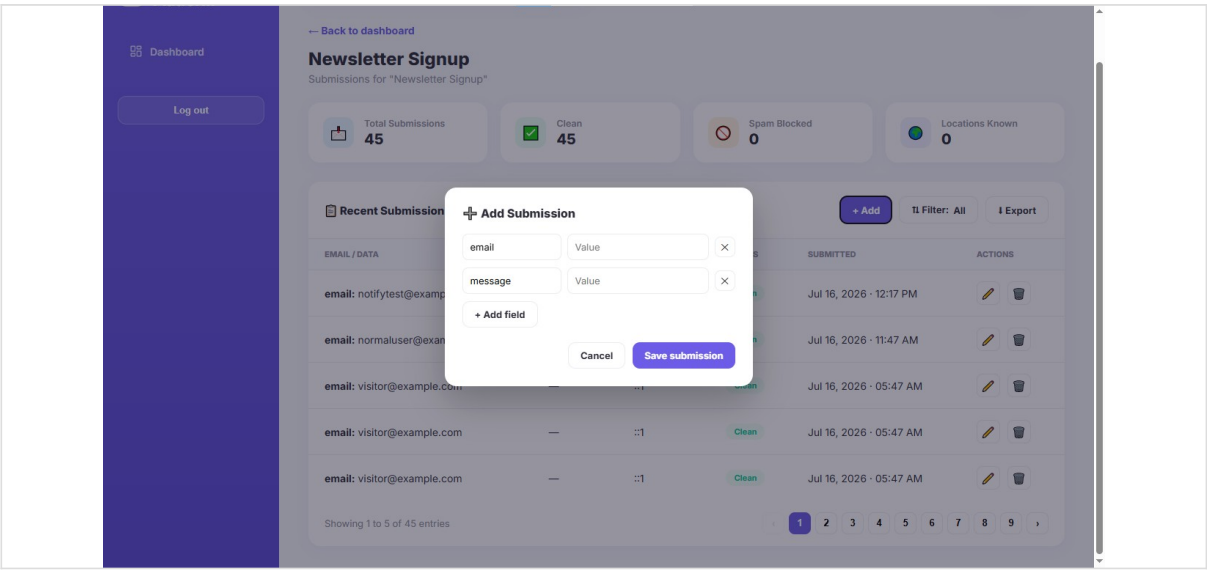

Figure 7: Adding a new submission manually via the Add dialog.

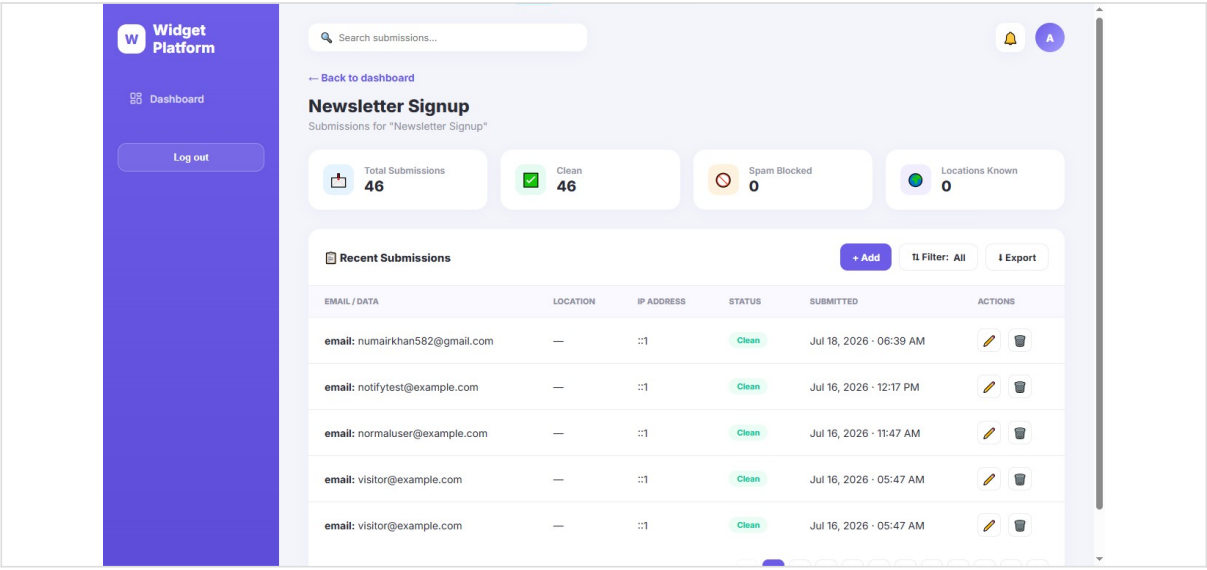

Figure 8: A newly submitted entry appearing instantly in the dashboard.

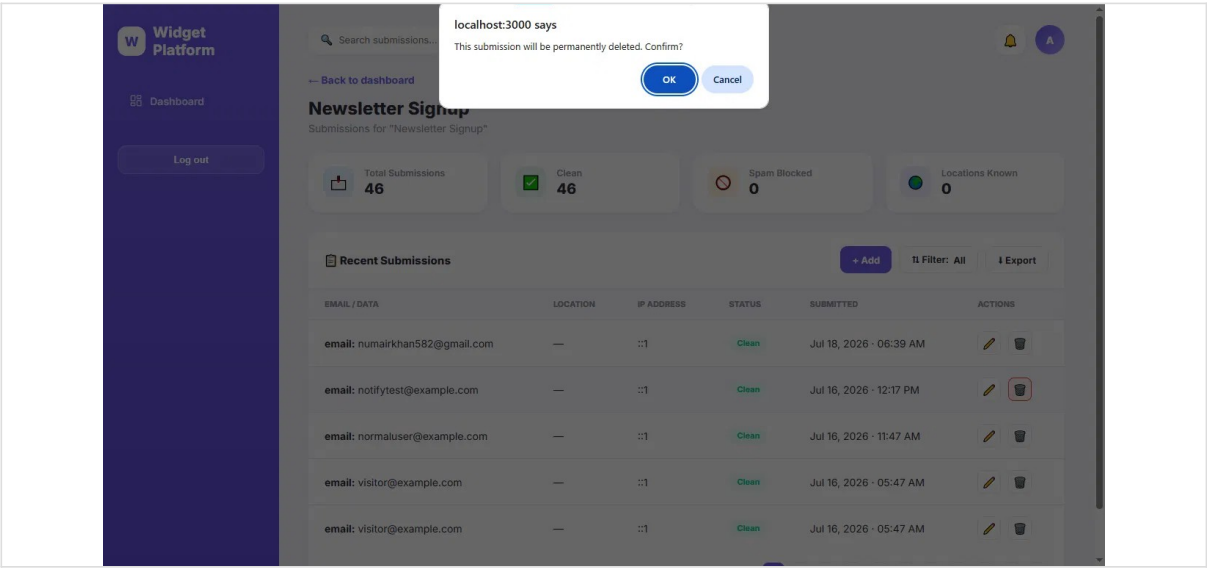

Figure 9: Delete confirmation prompt before a submission is permanently removed.

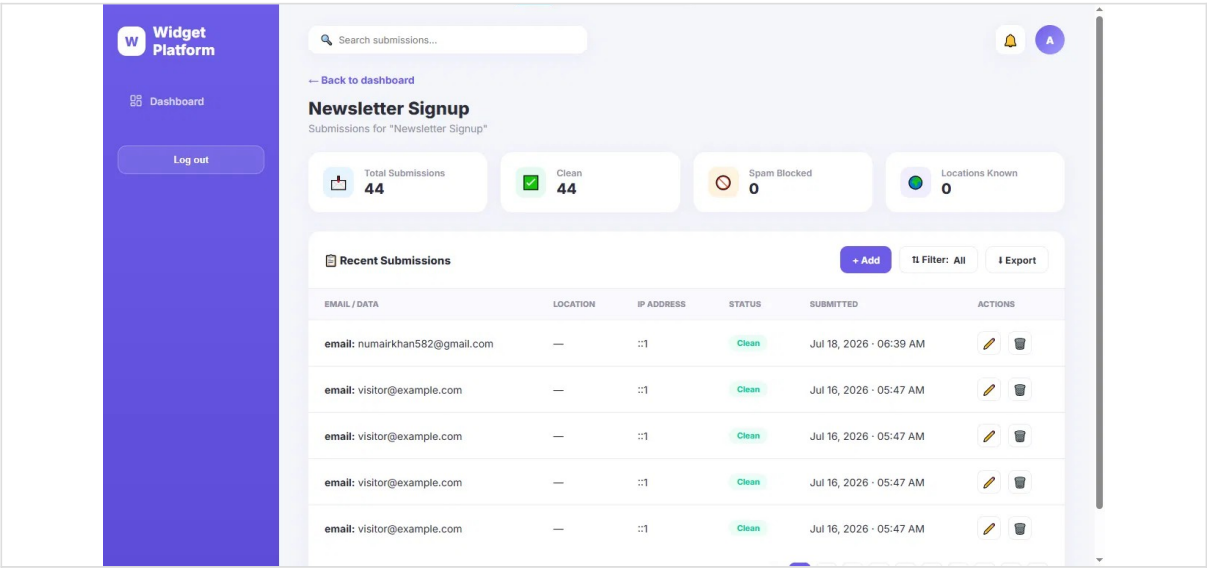

Figure 10: Submissions table immediately after a successful delete.

13.3 Database Tables (PostgreSQL)

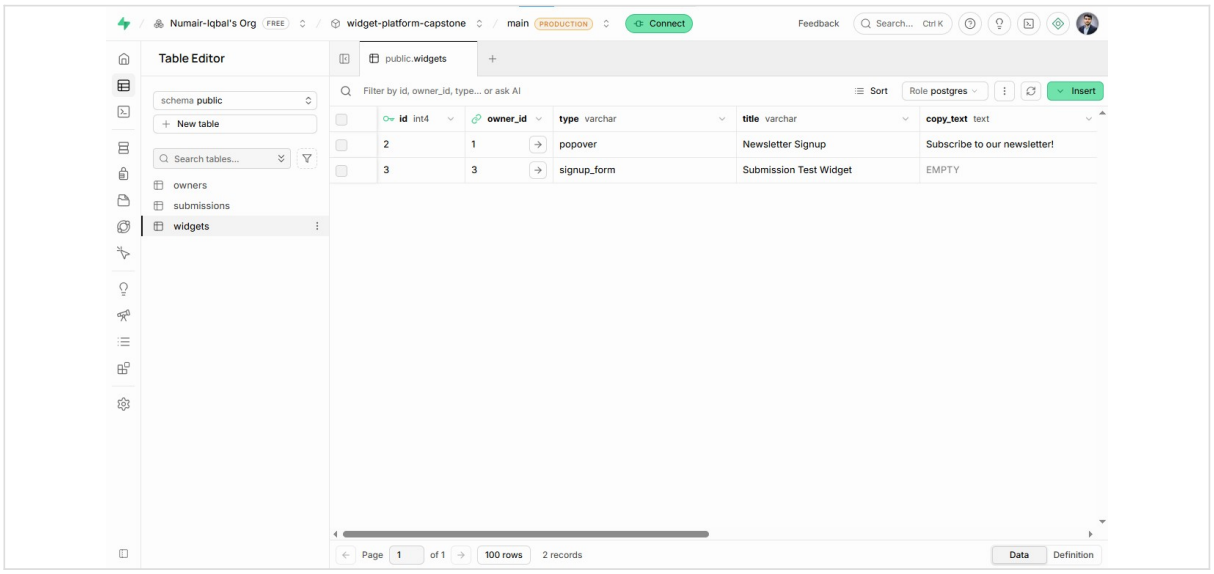

Figure 11: widgets table — one row per configured widget, scoped to its owner.

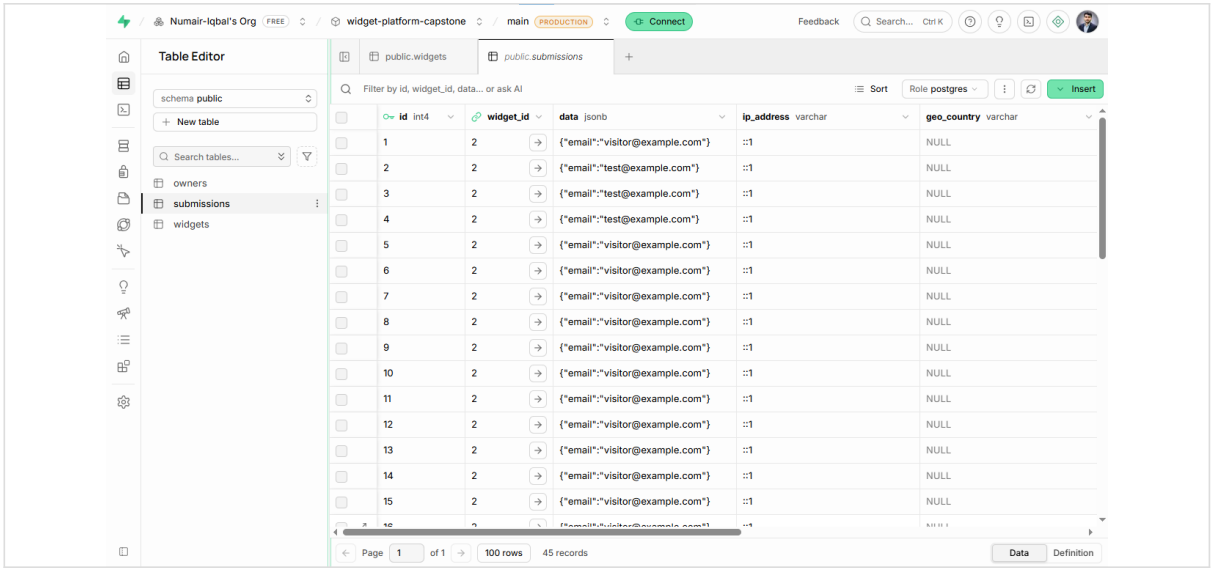

Figure 12: submissions table — raw visitor submissions with IP and geo fields.

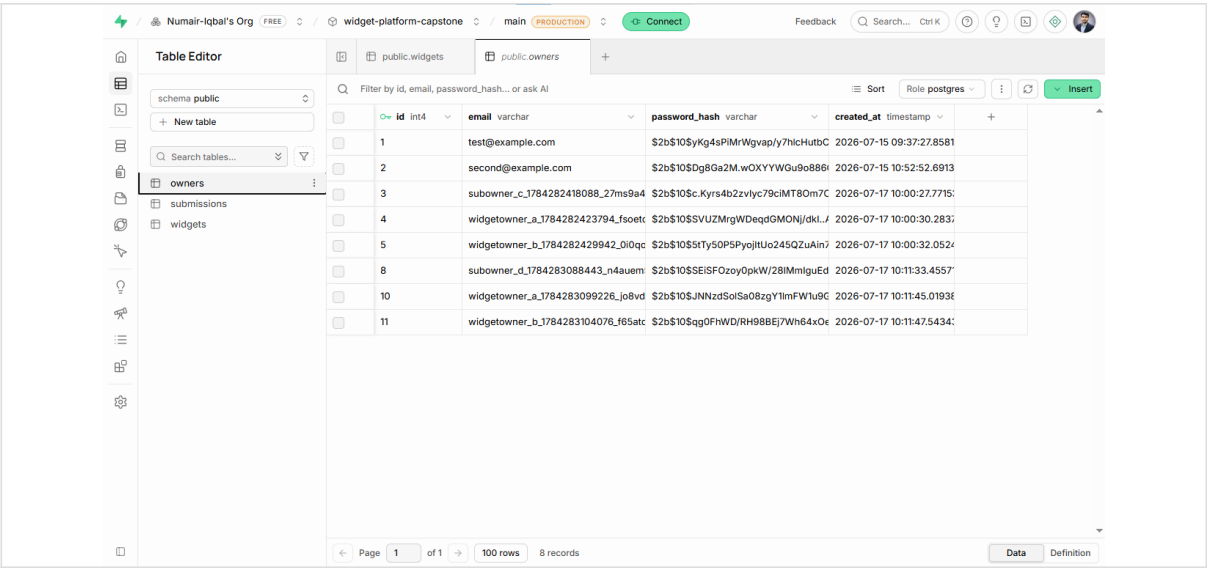

Figure 13: owners table — authenticated accounts with hashed passwords.

13.4 Embeddable Widget — Live on an External Site

These screenshots show the embed script running on a separate, external test page (customer-site-test/index.html), proving the widget renders and functions correctly outside of the platform's own domain.

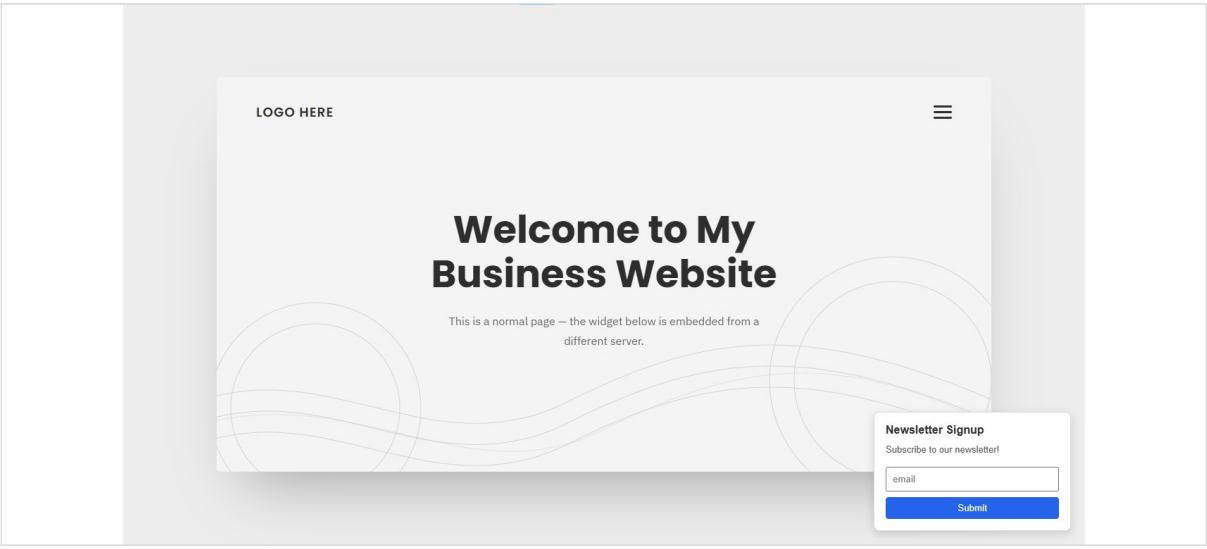

Figure 14: Embed script rendering the widget live on an external business website.

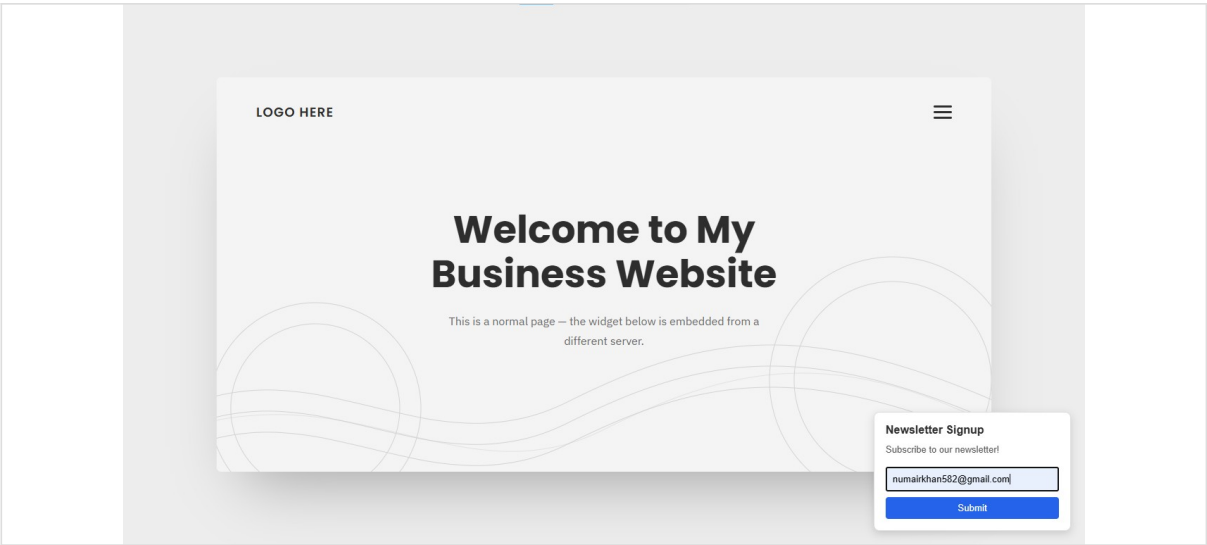

Figure 15: Visitor filling out the embedded widget form.

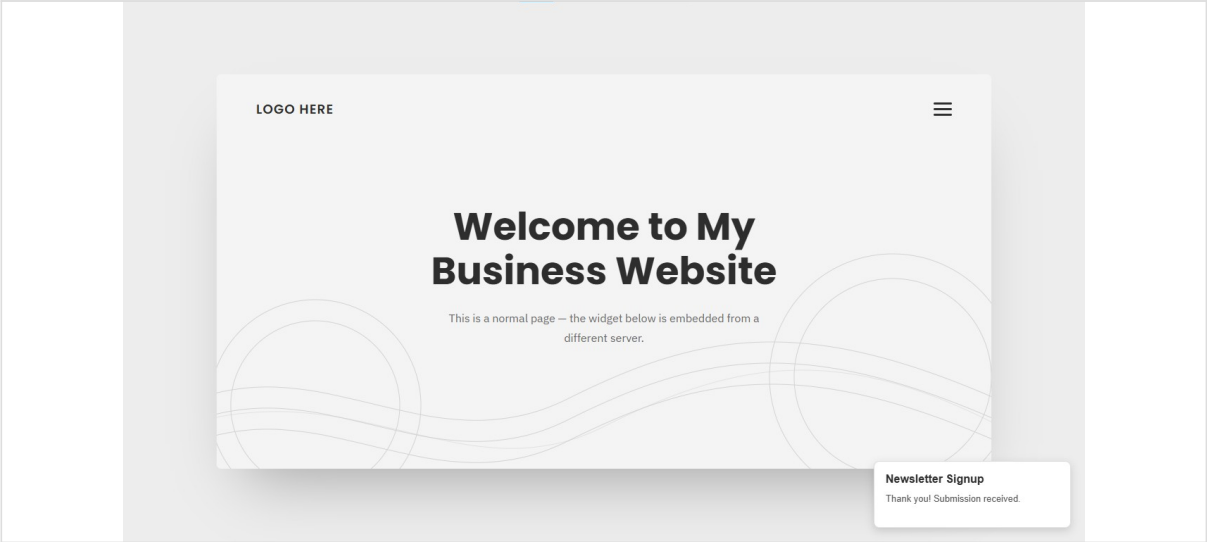

Figure 16: Confirmation message shown after a successful submission.

14. Documentation & Repository

- README.md — full project overview, setup instructions, and usage guide.
- Architecture diagram (docs/architecture-diagram.svg).
- 14+ screenshots covering login, dashboard, widget detail, database tables, passing tests, and the full embed-to-submission flow, stored under /screenshots.
- Complete, clean Git history with a single well-scoped initial commit (52 files, 9,874 insertions) followed by iterative development.

Public repository: github.com/Numair-Iqbal/embeddable-widget-platform

15. Challenges Encountered

- Express routing order initially caused the public submission route to be incorrectly blocked by an authentication middleware applied at the router level — resolved by re-ordering middleware registration.

- Ensuring true multi-tenant isolation required care at every query layer so one owner's data could never leak into another owner's dashboard.
- Balancing spam protection (honeypot, rate limiting) with a frictionless public submission experience for genuine visitors.
- Keeping OS-specific and environment files (desktop.ini, .env) out of version control before the first public push.

16. Conclusion

The Embeddable Widget & Lead-Capture Platform successfully delivers a complete, secure, and well-tested multi-tenant SaaS-style system, from database schema through to a live embeddable client and a polished admin dashboard. Every core requirement of the FlyRank capstone brief — widget management, embeddable delivery, secure submission handling, enrichment, spam protection, and reporting — has been implemented, tested, and documented.

17. Future Improvements

- Add role-based access control for teams with multiple dashboard users per owner account.
- Introduce webhook or email notifications on new submissions.
- Add analytics (conversion rate per widget, submissions over time).
- Containerize the application with Docker for simplified deployment.
- Add CI/CD via GitHub Actions to run the Jest suite automatically on every push.

18. References

- Node.js Documentation — nodejs.org/docs
- Express.js Documentation — expressjs.com
- PostgreSQL Documentation — postgresql.org/docs
- Jest Documentation — jestjs.io
- Project Repository — github.com/Numair-Iqbal/embeddable-widget-platform